

Sistemas Distribuídos — Aula 22

Sistemas Tolerantes a Falhas II: Replicação e Redundância · 29/10 · Prof. Leandro Borges

De detectar a mascarar: por que replicação é a estratégia central

Na Aula 21, olhamos para o diagnóstico de falhas em sistemas distribuídos: a taxonomia de Cristian, que organiza os tipos de falha por gravidade, e os mecanismos de detecção — heartbeat, ping, timeout e failure detectors não confiáveis — que permitem a um sistema perceber que algo deu errado. Diagnosticar uma falha, porém, resolve só metade do problema. Um failure detector que aponta corretamente um processo morto não tem nenhuma utilidade prática se o restante do sistema não tiver como continuar funcionando sem ele. Esta aula trata dessa segunda metade: como usar redundância e replicação para que uma falha, depois de detectada (ou mesmo antes), não vire indisponibilidade percebida pelo cliente.

A ideia central é a diferença entre detectar e mascarar uma falha. Detectar é constatar que um componente parou de responder corretamente; mascarar é impedir que essa parada afete o serviço entregue ao cliente, porque outro componente redundante assume no lugar dele. Um sistema com mascaramento de falhas é transparente a falhas: do ponto de vista de quem consome o serviço, ele continua respondendo corretamente mesmo que, por trás, uma ou mais réplicas estejam fora do ar. A replicação — manter múltiplas cópias de dados ou múltiplas instâncias de um processo — é a técnica mais geral para conseguir esse efeito, e retoma o tema de réplicas de dados já visto na Aula 20, mas agora sob a ótica específica de tolerância a falhas: quantas cópias manter, como organizá-las e como reagir automaticamente quando uma delas falha.

Redundância: hardware, dados e tempo

Replicar é uma forma de redundância: manter de propósito mais recurso do que o estritamente necessário, para que a perda de uma parte não derrube o sistema todo. Seguindo a apresentação de Tanenbaum e Van Steen, é útil distinguir três formas de redundância que aparecem em sistemas tolerantes a falhas. A redundância de hardware duplica componentes físicos — discos organizados em RAID, fontes de alimentação redundantes, servidores extras em standby — de modo que a falha de uma peça física seja absorvida por outra equivalente.

A redundância de dados, ou redundância de estado, mantém múltiplas cópias do mesmo dado ou do mesmo estado em processos diferentes, de forma que a perda de uma réplica não signifique a perda da informação: é essa a forma de redundância que sustenta diretamente as técnicas de replicação de processos discutidas mais adiante nesta aula. Já a redundância de tempo não duplica nada fisicamente: ela repete uma ação — retransmitir uma mensagem perdida, reexecutar uma operação que expirou por timeout — até que ela tenha sucesso. É particularmente útil contra falhas transientes, como uma perda momentânea de pacote na rede, em que o problema desaparece sozinho na segunda tentativa. As três formas não são excludentes: um sistema tolerante a falhas robusto costuma combinar as três, cada uma cobrindo um tipo diferente de falha.

Grupos de processos e comunicação em grupo confiável

Quando a estratégia é replicar processos, e não só dados, as réplicas precisam se organizar como um grupo de processos: um conjunto de processos que colaboram para oferecer, do ponto de vista de fora, um único serviço. A literatura distingue grupos fechados, em que só os próprios membros podem trocar mensagens entre si, de grupos abertos, em que processos externos ao grupo — como os clientes — também podem enviar mensagens para ele.

Para que a replicação de processos funcione corretamente, não basta que as mensagens cheguem às réplicas: elas precisam chegar de forma confiável e, em muitos casos, na mesma ordem para todos os membros. Fala-se em comunicação em grupo confiável quando uma mensagem enviada ao grupo é entregue a todos os membros que estão vivos, ou a nenhum deles — mesmo que o processo remetente falhe no meio do envio, e mesmo que algum membro do grupo falhe durante a própria entrega. Quando, além disso, essa entrega preserva a mesma ordem de mensagens para todos os membros, fala-se em multicast atômico (ou broadcast atômico). Essa garantia de ordem total é o alicerce sobre o qual se apoia a forma mais exigente de replicação de processos, tratada a seguir.

Réplicas ativas e réplicas passivas

Existem duas formas clássicas de organizar um grupo de réplicas de processo, e a escolha entre elas é uma das decisões de projeto mais importantes de um sistema tolerante a falhas. Na replicação ativa, também chamada máquina de estados replicada, todas as réplicas do grupo processam a mesma sequência de requisições, na mesma ordem — o que exige exatamente a comunicação em grupo totalmente ordenada descrita na seção anterior. Como todas partem do mesmo estado inicial e aplicam as mesmas operações na mesma ordem, todas chegam ao mesmo estado de forma determinística e simétrica, sem que nenhuma réplica precise de um papel especial de coordenação. A vantagem prática é que, se uma réplica falha, as demais já têm o estado atualizado: não há um processo de failover a esperar.

Na replicação passiva, ou modelo primary-backup, apenas uma réplica — o primário — processa efetivamente cada requisição; em seguida, ele propaga o novo estado (ou o registro da operação executada) para as réplicas backup. Esse modelo é mais simples de coordenar, porque não exige que todas as réplicas cheguem a um acordo sobre a ordem de cada requisição, apenas que o primário replique seu estado de forma confiável. O custo aparece quando o primário falha: um dos backups precisa assumir o papel de primário (failover), o que depende de detectar a falha do primário com confiança suficiente — retomando diretamente os mecanismos de detecção vistos na Aula 21 — e introduz um intervalo de tempo em que o serviço fica indisponível até a promoção do novo primário se completar.

Quorum systems: votação por maioria

Tanto na replicação ativa quanto na passiva, uma pergunta recorrente é: quantas réplicas realmente precisam participar de cada operação de leitura ou escrita? A resposta dos sistemas de quorum é: nem todas. Um quorum é um subconjunto de réplicas suficiente para autorizar uma operação, escolhido de forma que qualquer quorum de leitura e qualquer quorum de escrita sempre se sobreponham em pelo menos uma réplica. Formalmente, se NR é o tamanho do quorum de leitura, NW o tamanho do quorum de escrita e N o número total de réplicas, a condição $NR + NW > N$ garante que toda operação de leitura inclua pelo menos uma réplica que participou da escrita mais recente — e portanto enxergue o valor atualizado.

A variante mais usada na prática é o quorum de maioria: basta que NW seja maior que $N/2$ para garantir que duas escritas concorrentes nunca sejam aceitas sem que uma delas veja a outra, evitando atualizações perdidas. Esse mesmo raciocínio de maioria é o que determina quantas réplicas um sistema precisa manter para tolerar um número f de falhas. Para falhas de colapso, em que uma réplica simplesmente para de responder, $2f+1$ réplicas bastam: mesmo com f delas fora do ar, a maioria restante ainda consegue formar quorum e decidir. Para falhas bizantinas, em que uma réplica pode responder de forma arbitrária ou maliciosa — o problema descrito por Lamport, Shostak e Pease em 1982 —, é preciso um número maior, $3f+1$, porque o sistema precisa de réplicas corretas suficientes para superar em votos tanto as réplicas falhas quanto qualquer tentativa delas de confundir o resultado; a demonstração

formal desse limite foge ao escopo desta aula, mas o resultado é uma referência padrão na literatura de tolerância a falhas bizantina.

Próxima aula

Hoje vimos como a replicação, sustentada por redundância de hardware, de dados e de tempo, por comunicação em grupo confiável e por sistemas de quorum, permite mascarar falhas em vez de apenas constatar-las. Mas as próprias réplicas eventualmente falham e, quando voltam a funcionar, chegam com um estado desatualizado ou perdido. A Aula 23 fecha o bloco de três aulas sobre tolerância a falhas tratando exatamente disso: estratégias de recuperação de estado depois de uma falha e um estudo de caso real de sistema tolerante a falhas em produção.

Referências

TANENBAUM, Andrew S.; VAN STEEN, Maarten. Distributed Systems: Principles and Paradigms (edição atual: Distributed Systems). Pearson. Capítulo sobre tolerância a falhas: redundância, réplicas ativas e passivas, grupos de processos e quorum systems.

COULOURIS, George et al. Sistemas Distribuídos: Conceitos e Projeto. 5ª ed. Porto Alegre: Bookman. Capítulo sobre replicação: comunicação em grupo confiável, consistência de réplicas e tolerância a falhas.

LAMPORT, Leslie; SHOSTAK, Robert; PEASE, Marshall. "The Byzantine Generals Problem". ACM Transactions on Programming Languages and Systems, v. 4, n. 3, 1982. Artigo que introduz o problema das falhas bizantinas e a base para o limite de $3f+1$ réplicas.